

EECS 498 · AASE · FALL 2026

How Aider works

LAB01 · DESIGN YOUR PAIR-PROGRAMMER · SEP 14–15, 2026

Schedule

1. Aider deep dive
2. Hackathon 1
3. The next assignment
4. Office hours

The harness

Aider is a *harness*: a program that stands between you and a model.

The harness	The model
Picks the text the model sees	Reads it
Decides what a reply may do	Writes a reply
Edits files, then commits	Nothing

Take one apart now, build one later.

What the model never touches

- Your disk
- Your shell
- Your last session

Text in, text out. The harness does the rest.

What writes, and what does not

Arranges context

`/add`

`/read-only`

`/drop`

`/tokens`

`/ask`

`/diff`

Changes your files

`/code`

`/undo`

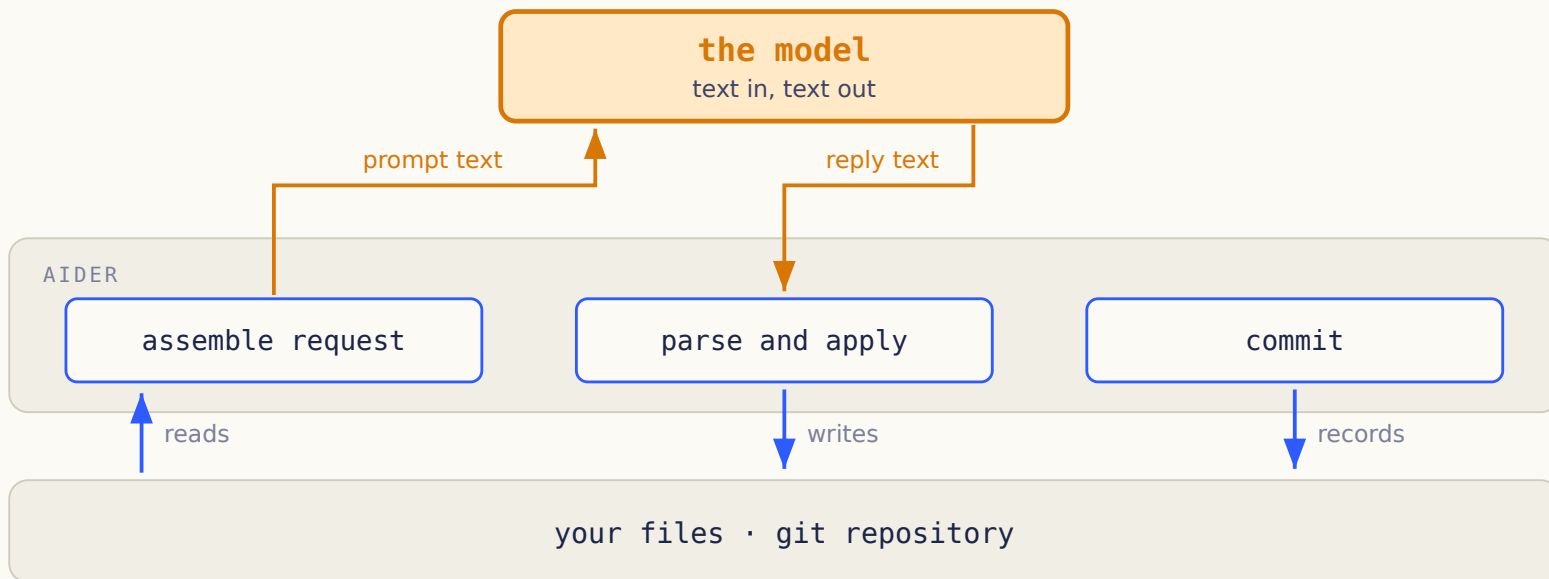
Bare text means `/code`. The default writes.

Context

WHAT THE HARNESS SENDS, AND WHAT COMES BACK

WHO DOES WHAT

Aider, the model, and the repo



The model touches no file.

What goes into the request

Fixed each turn	Instructions, added files, repo map
Grows each turn	Session history, your message

Context is a selection. A file you never added is not in it.

The repo map

```
package/store.py:
:
|class Store:
|    def add(self, title, tags=()) -> Task:
|    def remove(self, task_id: int) -> bool:
:
|    def search(self, text, tag=None) -> list[Task]:
:

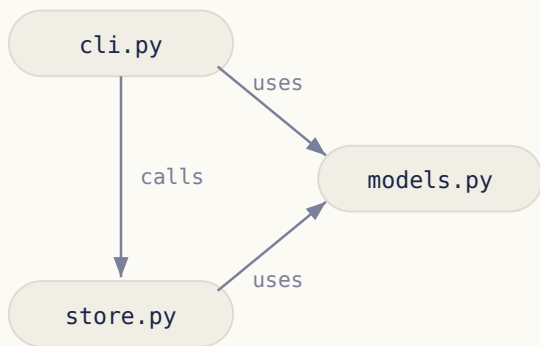
package/models.py:
|@dataclass
|class Task:
:
```

Signatures only. `:` is what was cut.

Files you never added.

Where the repo map comes from

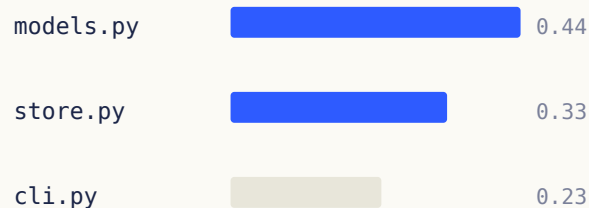
1. parse every file for what it defines and what it uses



models.py is used by both, so it wins

PageRank, rendered until the budget ends.

2. rank by what points at them



budget ends here

cli.py is left out entirely

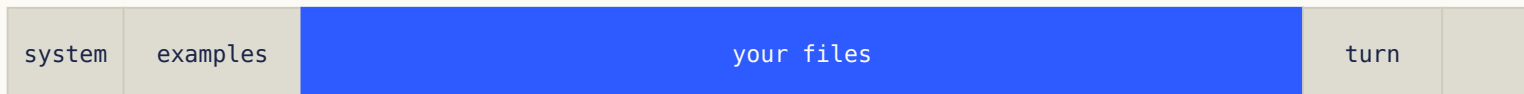
COST

What the map costs

repo map on



repo map off



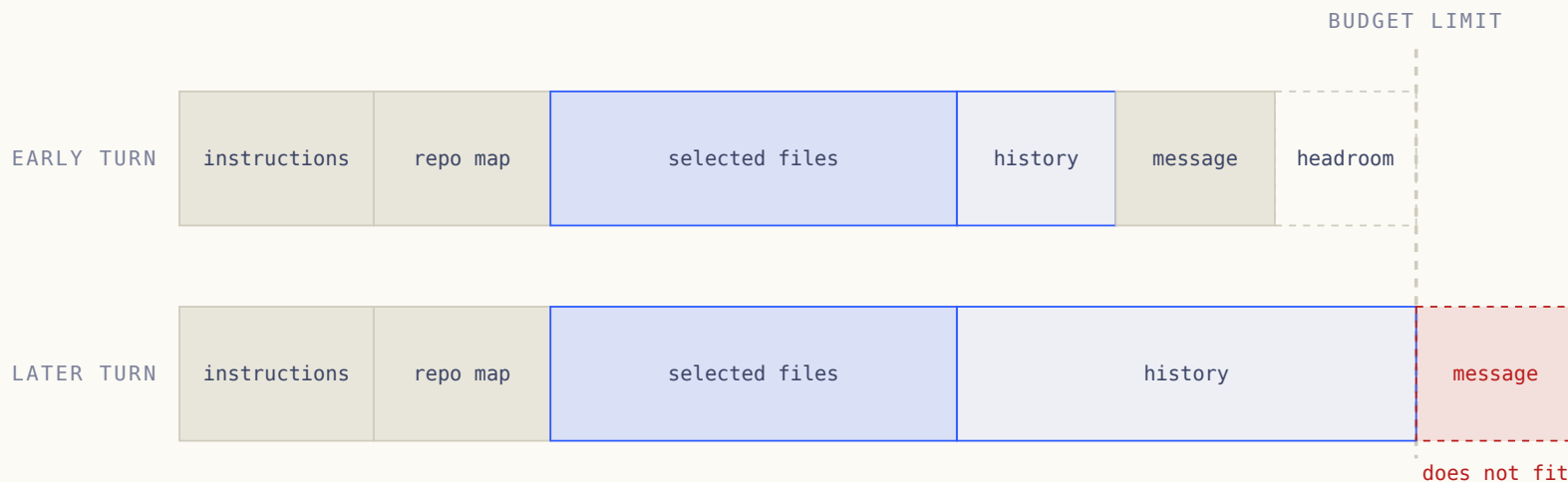
0

the whole context window

Same window either way.

On a small model the map can outweigh the files.

One shared token budget



One budget: history, map, and files.

THE REPLY

Edit-shaped replies

```
path/to/file.py
<<<<<<< SEARCH
the lines exactly as they appear on disk
=====
the lines that replace them
>>>>>>> REPLACE
```

One block, one file. Nothing forces the set to be complete.

THURSDAY SEPTEMBER 24

Hackathon 1

- Two hours, your own project
- Prompt revealed in the room
- Submit before you leave
- Graded separately from the build

Build due **Tuesday September 29, 11:59 PM.**

The pair-programmer

WHAT STAGE 1 ASKS FOR

A harness from scratch

Everything in the last half hour, written by you.

It holds	A set of files you chose
It asks	A model endpoint you configure
It writes	Only after you approve

Aider without Aider. Same job, your code.

Specification before implementation

You write the requirements before you write the code, and the commit history has to show it.

System design	Once, before implementation
Increment specs	One per change, before it

No specification is supplied.

The design artifacts

Artifact	Covers
Your requirements	Config, run contract, seven features
Increment specs	Ordered, testable, context scoped
Three diagrams	Components, sequence, edit lifecycle

Specification and diagrams

Criterion	Points
Requirements and acceptance criteria	10
Architecture and interface design	10
Increment specs	10
The three diagrams	4 / 3 / 3

Demonstrated behavior and verification

Criterion	Points
The seven features	20
Behavioral tests	8
Safety and recovery tests	7
Integration and reproducibility	5

WHAT YOU KEEP

The evidence trail

Commits	Specs before the code they describe
Sessions	Which context, which model, what happened
Reconciliation	Where the design changed, and why

Development evidence and reconciliation

Criterion	Points
Spec-first history	5
Deliberate Aider use	5
Diagnosis and reconciliation	5
Operating instructions	3
Model disclosure and index	2

`RUBRIC.md` carries every row.

Model rules for specifications

- Drafting, critique, diagrams, code, tests
- 9B recommended, 4B permitted
- Secondary models recorded too
- Any compatible host allowed

A different provider is not a different model.

What comes next

ONE REPOSITORY, GROWN UNTIL DECEMBER

THROUGH DECEMBER

One repository to December



Nothing is thrown away. Each phase grows the last.

WHERE THIS IS GOING

What an agentic CLI adds

Aider	An agentic CLI
You frame a bounded task	You hand over a broad one
You select the files	It discovers them
A fixed sequence of steps	The model picks the next tool
You check between tasks	Permissions bound it

BY DECEMBER

The harness by December

You will have built the kind of tool this lab took apart: a coding agent in the shape of Claude Code or OpenClaw, running against a model you configure, grown from the pair-programmer you start on Monday.

Today you drive one. In December you will have written one.

Office hours

THE REST OF THE SESSION IS YOURS

Questions?